

Due Nov 22 11:59pm

Submit your work by committing the hw4 directory of CNETID-cs144-aut-24 svn repository. Note that hw4 directory contains link.txt and hw4.txt, where link.txt is for you to check the homework questions in canvas, and hw4.txt is for you to write down answers (or you can use PDF file format (e.g., hw4.pdf) for your answers, PDFs of scanned handwritten pages must not exceed 6 megabytes) No other file formats or filenames are acceptable, and no files besides hw4.txt or hw4.pdf will be graded. Not following directions will result in losing points.

Related lecture: os2 (System Calls and Processes), os3 (Fork and signals)

(1) (20 points)

Some race conditions plague the following code, which can be compiled and run on the CSIL Linux machines (with “gcc -m32 -o race race.c; ./race”).

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <signal.h>
4
5  pid_t forkret = 0;
6
7  void handler1(int sig) {
8      printf("(%) handler1\n", getpid());
9      if (!forkret) {
10         printf("(%) seeya\n", getpid());
11         exit(0);
12     }
13 }
14
15 int main() {
16     printf("(parent) hello -----\\n");
17     signal(SIGUSR1, handler1);
18     if ((forkret = fork()) == 0) {
19         printf("(child) hello I'm %d\\n", getpid());
20         kill(getppid(), SIGUSR1);
21         while (1) {};
22     } else {
23         printf("(parent) child is %d\\n", forkret);
24         kill(forkret, SIGUSR1);
25         wait(NULL);
26         printf("(parent) child done\\n");
27     }
28     exit(0);
29 }
```

A. Reading the code, and seeing that the child and the parent both send a SIGUSR1 signal to each other after the fork, one might expect output like (knowing that PIDs differ between runs):

```
1  (parent) hello -----
2  (parent) child is 101
3  (child) hello I'm 101
```

```
4 (100) handler1
5 (101) handler1
6 (101) seeya
7 (parent) child done
```

Give an account of each numbered line of output, by writing

- the number of the output line (no need to reproduce the output line itself), and then all in the same line:
- “P” or “C” for whether it came from the parent or the child process, respectively
- the line number from the C source code whose execution produced the output
- the value of `forkret`

B. One possible output of the program is:

```
1 (parent) hello -----
2 (parent) child is 101
3 (child) hello I'm 101
4 (101) handler1
5 (101) seeya
6 (parent) child done
```

Give an account of each line of output like in part A, and then answer: what is the race condition that leads to this output instead of that in part A? That is, what are the steps in each of the child and parent processes that are in a race with each other, so that depending on which one finishes first, the output is either that from part A or part B? (We’re using “step” here instead of “instruction” because in this case we can describe the race condition without getting down to the level of assembly, but we may need a finer level of granularity than lines of code.)

C. Another possible output of the program is:

```
1 (parent) hello -----
2 (parent) child is 101
3 (101) handler1
4 (101) seeya
5 (parent) child done
```

Like part B: give an account of each output line, and describe the race condition and why it leads to this output.

D. Another possible output of the program is:

```
1 (parent) hello -----
2 (child) hello I'm 101
3 (100) handler1
4 (100) seeya
```

Like part B: give an account of each output line, and describe the race condition and why it leads to this output.

REMINDER - STARTING IN 2022, WE HAVE STRICTER POLICIES ON PROJECT AND HOMEWORK SUBMISSIONS:

Please triple check that you have submitted the right file(s), the right filename, the right content, the right format, and the right version.

We recommend submitting 15 minutes before the deadline (or even earlier) so you have 15 minutes to double/triple check your submission.

We will not accommodate incorrect or late submissions. 11:59:59pm is the hard deadline.